ISO - Final

Docente: **Ing. Juan Cruz Martinez Luquez.**
Alumno: **Antrichipay, Milton Daniel.**

1. Virtualización vs Contenedores

A la hora de comparar la **virtualización** y la **contenerización**, lo primero que se debe hacer, es definir y diferenciar los rasgos principales de cada uno, en primera instancia, la **virtualización** se define como la **tecnología que permite crear recursos informáticos de forma virtual**, recursos como sistemas operativos (S.O), hardware, almacenamiento, redes, etc. **No se centra únicamente en la creación de entornos completos de ejecución** como vienen siendo las **maquinas virtuales**, sino que en la **virtualización de recursos informáticos** de diversos fines, como puede ser la creación de VLANs. Por otro lado, la **contenerización** o los **contenedores**, si **se basan en la creación de entornos de ejecución aislados e independientes**. La creación de contenedores busca crear un espacio de ejecución **controlado**, **aislado** y **potable** en donde solo se contenga lo necesario para ejecutar procesos bien definidos. Si bien, no tienen un punto de comparación ya que se trata de conceptos y objetivos distintos, podemos hacer una comparación entre los contenedores y **Maquinas Virtuales** (MV).

Maquinas Virtuales vs Contenedores

> **Nota**
>
> En este contexto se entiende como "**host**" a la maquina en la cual se despliegan tanto MVs como contenedores.

Tanto las **MV** como los **contenedores** tienen un mismo fin, brindar un entorno de ejecución **controlado**, **potable** y **aislado**, pero la forma en la que estas tecnologías llegan a ese fin es distinta.

La diferencia de estas tecnologías radica en su funcionamiento, por un lado se tiene la virtualización completa de un computador (o la paravirtualización), y por el otro tenemos el uso de un **engine** que gestiona todo, junto a las funciones **namespaces** y **cgroups** del kernel perteneciente al host.

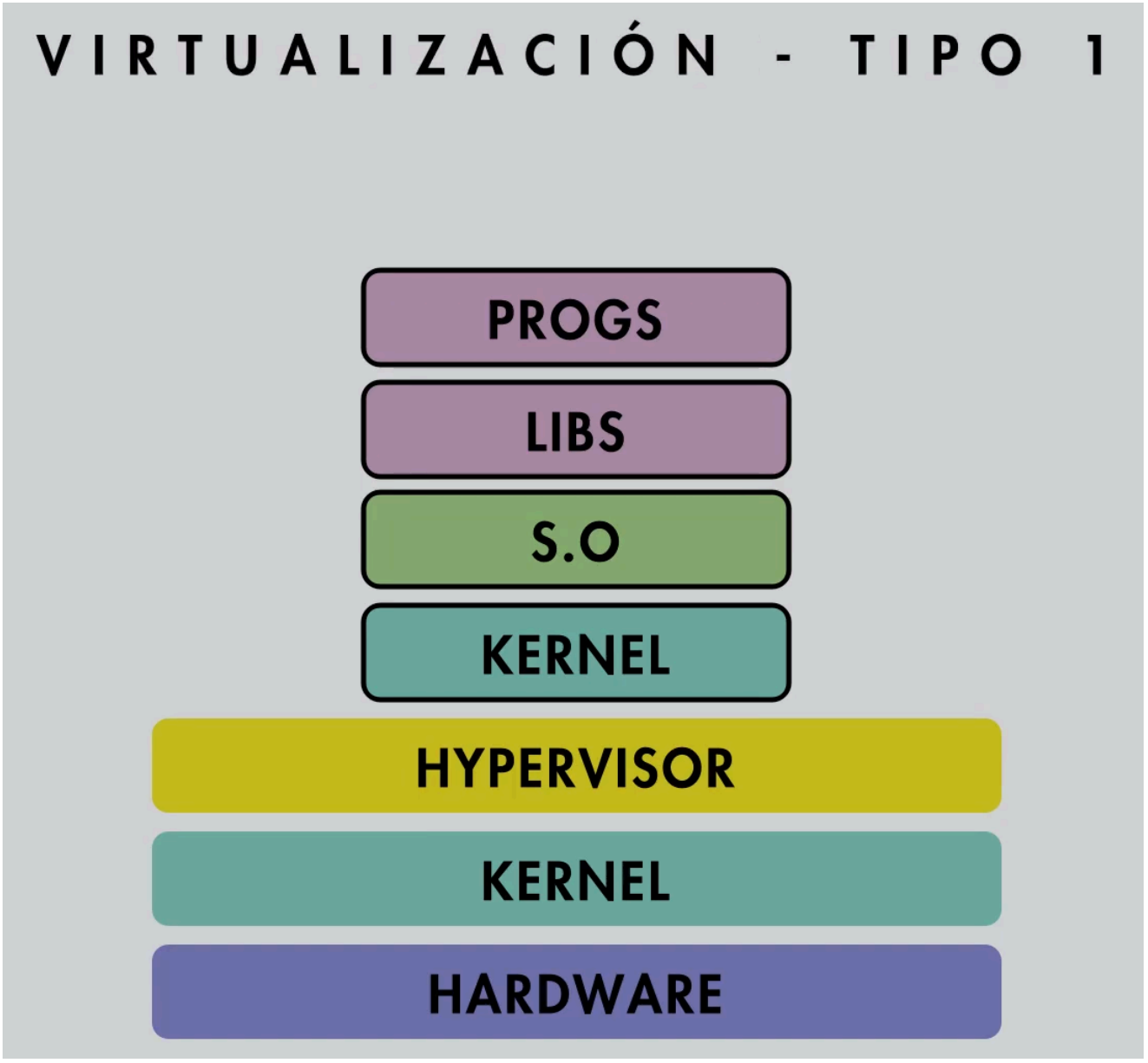
Maquinas virtuales:

Las MV enfatizan su solución en la virtualización completa (o la paravirtualización) de un computador a través de un **Hipervisor**. El Hipervisor es el encargado de gestionar todas las

maquinas virtuales a nivel de procesos y de recursos de hardware, esto lo hace aplicando uno de los siguientes esquemas:

Tipos de Hipervisor

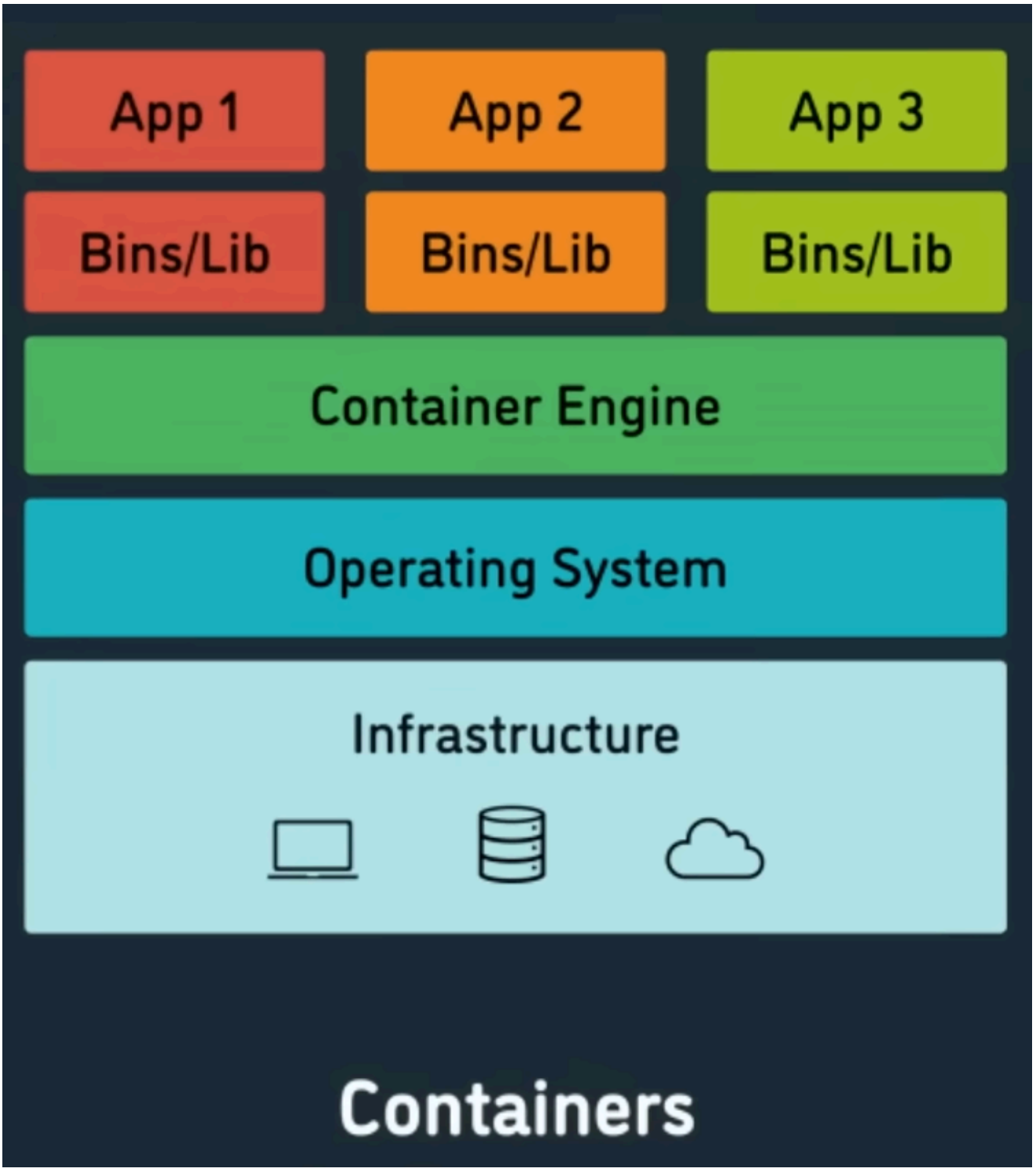
- Hipervisor de tipo uno: Se caracteriza por instalarse y funcionar directamente sobre hardware.
- Hipervisor de tipo dos: Se caracteriza por instalarse y funcionar sobre un S.O host.



Bajo estos esquemas, el Hipervisor busca crear un entorno de ejecución compuesto de su propio kernel, S.O completo, librerías y hardware virtualizado. Si bien esto es una solución completa y funcional, la misma lleva a que en cada MV se tengan los mismos componentes que ya se encuentran en otra, o los que ya puede posee el mismo host (kernel y S.O), volviéndose una solución muy costosa para algunos fines.

Contenedores:

Los contenedores plantean un medio distinto con el cual crear entornos de ejecución, en lugar de virtualizar hardware y computadores completos, utilizar el kernel que ya tiene el host para asignar recursos y aislar procesos, para esto se plantea el siguiente esquema:



Bajo este esquema, se establece que no se virtualizarán computadores completos, sino que cada contenedor tendrá la aplicación que se quiere contenerizar, las dependencias de la misma y un userspace básico del S.O que utilizara en el contenedor, trabaj ando en conjunto con el container engine (encargado de gestionar los diversos contenedores). La idea principal es no usar un sistema operativo completo, osea no usar un sistema operativo con kernel, sino utilizar el kernel del host junto algunos programas base del sistema operativo, para esto, los contenedores utilizan principalmente dos funciones del kernel el **namespaces** para ocultar procesos y del **cgroups** para asignar y limitar recursos.

Debido a este planteamiento, los contenedores consiguen un entorno de ejecución mas ligero, rápido y portable.

Comparativa

Característica	Máquina Virtual (MV)	Contenedores
Componente Clave	**Hypervisor** (tipo 1, tipo 2).	**Container Engine** + Kernel de Linux.
Kernel	Cada MV tiene su propio Kernel.	Todos comparten el Kernel del host.
Aislamiento	**Fuerte:** Nivel hardware. Muy seguro.	**Moderado:** Aislamiento de procesos (Namespaces).
Consumo Recursos	Alto (OS completo + Hardware emulado).	Muy bajo (Solo la App + Librerías).
Tiempo de Arranque	Minutos (debe bootear el OS).	Milisegundos/Segundos (es un proceso).
Portabilidad	Difícil (archivos `.vdi` o `.vmdk` pesados).	Alta (Imágenes ligeras y estandarizadas).
Uso Ideal	Correr varios OS para pruebas o usos personales.	Microservicios, CI/CD, despliegue rápido.
Overhead de RAM	**Muy Alto:** Reserva fija para el OS invitado.	**Mínimo:** Solo consume lo que usa la App.
Overhead de Red	**Medio:** Pasa por drivers virtuales.	**Bajo:** Bridge virtual.
Overhead de Disco	**Alto:** Archivos de disco rígido virtual (GBs).	**Bajo:** Capas de imagen compartidas (MBs).

2. ¿Qué es Docker? ¿Cuáles son sus componentes principales?

Docker es un proyecto opensourse muy utilizado que busca brindar una tecnología que facilita la creación y administración de contenedores. Sus componentes principales son:

- **Docker engine**: Motor a través del cual se administra todos los contenedores.
- **Imágenes de docker**: Imágenes de solo lectura que contiene toda la configuración del contenedor.
- **Dockerfile**: Archivo de escritura en el cual se define lo que tendrá el contenedor (genera una imagen de docker).
- **Docker Service/Daemon**: Un servicio que corre en segundo plano y permite interactuar con docker.
- **Volúmenes**: Es el espacio de almacenamiento que se le da a un contenedor (no esta dentro del contenedor, sino que este escribe en un espacio finito del host).
- **Docker Compose**: Archivo .yaml que permite la gestión de múltiples contenedores de forma simultanea a través de sus imágenes.

- Redes: Redes virtuales que comunican los contenedores a través de la resolución de nombres (DNS).
- **Contenedor** docker: Contenedores creados a través de las imágenes de docker.

3. Respecto a Docker, defina y explique: Dockerfile Volúmenes ,Redes ,Docker Compose.

Dockerfile

Docker file es un archivo en el cual se definen los pasos necesarios para construir la imagen de un contenedor. Al momento de crear el dockerfile de una aplicación, lo primero que se debe hacer es definir la imagen base que se utilizara (`FROM nombre_imagen:version`), esta imagen suele contener el sistema operativo y las dependencias generales que la aplicación necesita, luego, de ser necesario, se debe descargar e instalar las dependencias faltantes, esto se hace a través del instrucción `RUN` , `RUN` se encarga de definir los comandos que se ejecutaran dentro del contenedor al momento de construirlo, permitiendonos descargar e instalar paquetes. Con las dependencias suplidas, se debe definir y configurar la aplicación que correrá en el contenedor, para esto se utilizan las instrucciones:

- `COPY` : Con la instrucción `COPY` , lo que haremos es indicar de donde se obtendrá la aplicación a contenerizar y en que lugar del contenedor se colocara. (`COPY ruta/local ruta/contenedor`)
- `ADD` : Esta instrucción funciona igual que `COPY` , pero con la diferencia de que permite obtener la aplicación desde una URL. (`ADD url.com.ar ruta/contenedor`)
- `WORKDIR` : Esta instrucción nos ayuda mejorar la legibilidad del dockerfile, esta permite definir el directorio del contenedor sobre el cual se trabajaran las siguientes instrucciones. por ejemplo:

```
WORKDIR /app
COPY . .
RUN npm install
```

- `ENV` : Muchas veces se necesita configurar una aplicación a través de sus variables de entorno, asi que para solventar esto, se nos da la instrucción `ENV` , esta instrucción nos permite darle valores a esas variables. (`ENV VARIABLE=valor`)
- `EXPOSE` :Esta instrucción se utiliza para documentar los puertos en los que la aplicación va a escuchar. (`EXPOSE puerto` - `EXPOSE puerto/protocolo`)

Volúmenes

En principio los contenedores no pueden persistir información, una vez que el contenedor se apague todos sus datos se perderán. Para solucionar esto, se le asignan volúmenes a aquellos contenedores que lo necesitan, los volúmenes son un espacio de memoria

perteneciente la maquina host y en el cual los contenedores asignados a ese volumen tendrán acceso.

Redes

Las redes de docker brindan una forma de comunicarse entre contenedores y entre contenedores y el host, por ejemplo, en el caso de desplegar un sistema compuesto por un backend y un frontend, estos deben comunicarse entre si, a la vez que deben comunicarse con el exterior, para esto se establece una red virtual entre el backendo y el frontend (con su propio DNS), y luego con el exterior a través del mapeo de puertos. Docker no solo se queda en esto, también nos brinda distintos tipos de redes, cada una con su propio funcionamiento y objetivo, por ejemplo:

- **Bridge** (Puente): Es la red por defecto. Ideal para aplicaciones que corren en un solo host. Crea una red privada virtual donde los contenedores pueden hablar entre sí usando nombres de dominio internos (DNS), pero están aislados del exterior a menos que mapees puertos.
- **Host**: Aquí el contenedor elimina el aislamiento de red. Utiliza directamente la IP y los puertos del host. Si el contenedor abre el puerto 80, estará abierto en el host inmediatamente. Es muy rápido (bajo overhead), pero menos seguro y propenso a conflictos de puertos.
- **Overlay:** Esta es la red para sistemas distribuidos. Permite conectar contenedores que están en **diferentes máquinas físicas** (nodos de un clúster). Es la base de Docker Swarm y permite que el backend en el Servidor A hable con el frontend en el Servidor B como si estuvieran en el mismo equipo.
- **Macvlan:** Permite asignar una **dirección MAC** propia a un contenedor, haciendo que aparezca como un dispositivo físico real en tu red local. Es útil para aplicaciones antiguas (legacy) que necesitan estar directamente en la red física por temas de monitoreo o protocolos especiales.
- **None:** El contenedor no tiene interfaces de red externas. Solo tiene la interfaz de *loopback* (localhost). Se usa para tareas de procesamiento de alta seguridad o scripts que no deben tocar la red bajo ninguna circunstancia.

Docker Compose

Docker Compose es una herramienta que brinda docker para la definición y gestión de aplicaciones de múltiples contenedores. Este permite crear un archivo .YAML en el cual se definan los contenedores (utilizando imágenes ya generadas), volúmenes, redes, puertos, etc. Para implementar un compose se deben definir:

- **Version**: Define la versión del compose.
- **Servicios**: Son aquellos contenedores que forman parte del sistema y son contruidos por el archivo compose.yaml, cada uno de estos tiene definido la imagen del contenedor y todo lo que este requiera (puertos, volumen, comandos, etc).

- **depends_on**: Define en que orden se levantaran los componentes.
  Por ultimo se definen los secretos y variables de entorno necesarias.